

Подготовка к коллоквиуму по модулю 3: SFML, ООП в C++ и паттерны проектирования

Краткое резюме

Вопросы модуля делятся на три «оси»: событийно-ориентированное программирование и ввод/рендер в SFML ¹; механика наследования и динамического полиморфизма в C++; и инженерные практики (принципы проектирования, Design Patterns: Elements of Reusable Object-Oriented Software ² -паттерны, UML). Точка «наибольшего риска» на коллоквиуме — не определения, а тонкости: где именно в SFML находится очередь событий и почему неправильно читать «не тот» член union в `sf::Event`; чем «реальное состояние клавиатуры» (`isKeyPressed`) отличается от событий `KeyPressed/KeyReleased`; как устроен цикл `clear/draw/display` и почему `display()` связан с двойной буферизацией; где именно может возникнуть UB при downcast через `static_cast`; что означает EBO (empty base optimization) и почему при множественном наследовании указатель «сдвигается»; как у polymorphic-объекта появляется скрытый указатель на vtable и почему это важно для `dynamic_cast / typeid`. ³

По принципам проектирования ожидают не «перечислить SOLID», а связать их с мотивацией: разделение по причинам изменения (SRP) и защита от изменений (OCP) исторически восходят к идее «скрывать решения, которые вероятно изменятся» (Parnas); современные конструкции (DIP и «предпочитайте композицию наследованию») часто реализуются конкретными паттернами (Strategy, Decorator, Factory Method и др.). ⁴

Кодовая опора отчёта начинается с репозитория на GitHub ⁵: (1) `SFML/SFML` как первоисточник API/архитектуры окон, событий, времени; (2) `RefactoringGuru/design-patterns-cpp` как база коротких runnable-примеров паттернов на C++17; дополнительно использованы официальные страницы документации SFML и материалы Refactoring.Guru ⁶ (в русской локализации там, где доступна). ⁷

Репозитории и первичные источники

Репозитории GitHub, задействованные как «основной кодовый корпус»

`SFML/SFML` — исходники и структура модулей Window/System/Graphics (основа: окно, контекст OpenGL, очередь событий, ввод, время, sleep, framerate limit и т.д.). ⁸

`RefactoringGuru/design-patterns-cpp` — набор «Conceptual»/«RealWorld» примеров GoF-паттернов на C++17; полезно как готовый «скелет» для воспроизведения кода на коллоквиуме. ⁹

(Фон/дополнение для GUI-виджетов) `texus/TGUI` — зрелая GUI-библиотека (исторически SFML-backend), полезна как ориентир того, «как выглядит» промышленная реализация кнопки/слайдера поверх рендера и событий. ¹⁰

Первичные/официальные тексты (после GitHub)

Официальные tutorиалы SFML по циклу рендера и событиям (обязательный `clear/draw/display`, скрытый буфер и double buffering; устройство `sf::Event` как union; правила использования `pollEvent`). ¹¹

Официальная документация SFML по вводу, событиям, координатным преобразованиям и времени (`Keyboard::isKeyPressed`, `Mouse::isButtonPressed`, `Event`, `RenderWindow::mapPixelToCoords`, `Time`, `sleep`, `ContextSettings`). ¹²

Базовые источники по C++-механике и ABI

`cppreference` как практичная «квазипервичка» для правил языка: `static_cast`, `dynamic_cast`, `virtual`, `override`, `typeid`, `std::type_info`, EBO. ¹³

Itanium C++ ABI ¹⁴ как спецификация де-факто для размещения объектов, vtable/RTTI и виртуального наследования в распространённых toolchain (важно: стандарт C++ не фиксирует layout, ABI фиксирует). ¹⁵

Принципы проектирования и паттерны

Статьи Robert C. Martin ¹⁶ (Clean Coder Blog) по SRP и OCP, связывающие принципы с историческими источниками и практикой. ¹⁷

Русскоязычные страницы Refactoring.Guru по категориям и примерам паттернов (поведенческие/структурные/порождающие; Strategy/Observer и др.). ¹⁸

Источники по настройкам автоповтора клавиатуры

Определения «delay until repeat» и «repeat rate» как параметров ОС (пример: Apple macOS UI). ¹⁹

Точные определения delay/interval для ХКВ (X11), показывающие, что «задержка до первого повтора» и «интервал между повторами» — системные параметры, а не часть SFML. ²⁰

Соответствие вопросов и материалов репозитория

Ниже — таблица «вопрос → какие репозитории/файлы использованы как опора». Там, где указывается путь, он соответствует общепринятой структуре репозитория (SFML: `include/SFML/...`; RefactoringGuru: `src/<Pattern>/Conceptual/...`); конкретные API-символы подтверждены официальной документацией.

Вопрос из PDF	Опорные репозитории	Ключевые файлы/артефакты кода
1a–1c (SFML: цикл, framerate, время, double buffering, события, координаты)	SFML/SFML	include/SFML/Window/WindowBase.hpp (pollEvent/waitEvent, key repeat), include/SFML/Window/Event.hpp (типы событий), include/SFML/Window/Keyboard.hpp, include/SFML/Window/Mouse.hpp, include/SFML/System/Time.hpp, include/SFML/System/Sleep.hpp, документация RenderWindow (mapPixelToCoords/mapCoordsToPixel), tutoriales graphics-draw, window-events ²¹
1d (GUI: draggable/button/slider)	SFML/SFML (+ ориентир texus/TGUI)	Реализация «с нуля» строится на Event, Mouse, mapPixelToCoords; как ориентир — документация tgui::Slider ²²
2a–2h (наследование, затенение, доступ, касты, slicing, layout, множественное/виртуальное MI)	(язык C++ + ABI)	cppreference: static_cast, dynamic_cast, EBO; Itanium ABI: data layout, vtable, virtual inheritance ²³
3a–3f (динамический полиморфизм, vtable, abstract, RTTI)	(язык C++ + ABI)	cppreference: virtual, override, typeid, std::type_info, dynamic_cast; Itanium ABI RTTI/vtable layout ²⁴
4a–4c (SOLID, «инкапсулируйте изменчивое», композиция вместо наследования, UML отношения)	Refactoring.Guru + статьи Martin + общие источники	SRP, ОCP у Martin; каталоги паттернов на Refactoring.Guru; связки отношений UML (см. раздел UML ниже) ²⁵
5a–5f, 6a–6f, 7a–7e (паттерны)	RefactoringGuru/design-patterns-cpp + Refactoring.Guru	Категории и определения: Refactoring.Guru; код-скелеты: src/<Pattern>/Conceptual/* в репозитории ²⁶

Событийно-ориентированное программирование и SFML

Полный список вопросов блока

Из module3_questions.pdf извлечены формулировки (раздел 1): SFML (цикл, setFrameRateLimit, sf::Time/sf::Clock/sf::sleep, double buffering, антиалиасинг, координатные системы и mapPixelToCoords/mapCoordsToPixel), проверка нажатий (Keyboard::isKeyPressed, Mouse::isButtonPressed и отличие от событий), события/очередь/цикл обработки (sf::Event, Closed/Resized/KeyPressed/..., координаты мыши,

repeat delay/rate), и реализация простых GUI-элементов (Draggable, Button, Slider). (Текст вопросника — пользовательский источник; ниже ответы опираются на cited-источники.)

Ответы по пунктам раздела 1

Библиотека SFML, основной цикл, framerate, время, sleep, double buffering

Короткий ответ: типичный цикл SFML — это (1) обработать все события из очереди окна; (2) обновить состояние; (3) выполнить `clear`, затем `draw`, затем `display`. Ограничение FPS (`setFramerateLimit`) работает через задержку после `display` (в конце кадра). Двойная буферизация означает, что рисование идёт в скрытый буфер, который становится видимым при `display()`. ²⁷

Подробно: ядро цикла `clear/draw/display` и двойная буферизация.

В SFML «рисуют не прямо в окно», а в скрытый буфер; `display()` «забирает то, что нарисовано с прошлого `display`, и показывает это на окне», копируя/переключая буферы (double buffering). Поэтому «рисовать один раз и много раз вызывать `display`» даёт «странные результаты» — классическая ошибка новичка. ²⁸

Мини-пример (канонический цикл):

```
sf::RenderWindow window(sf::VideoMode(800, 600), "Demo");
while (window.isOpen()) {
    sf::Event event;
    while (window.pollEvent(event)) {
        if (event.type == sf::Event::Closed) window.close();
    }
    window.clear();
    // window.draw(...);
    window.display();
}
```

Смысл: «каждую итерацию полностью перерисовываем кадр»; это согласуется с тем, как устроено современное GPU-рендеринг-железо и как SFML организует буферы. ²⁹

`setFramerateLimit`, `sf::Time`, `sf::Clock`, `sf::sleep`.

`sf::Time` — оболочка для значения длительности (секунды/миллисекунды/микросекунды), с методами `asSeconds/asMilliseconds/asMicroseconds`. ³⁰

`sf::Clock` измеряет прошедшее время (`getElapsedTime`) и умеет «перезапускаться» (`restart`). ³¹

`sf::sleep(Time)` усыпляет текущий поток на заданную длительность. На уровне заголовка это объявлено как `void sleep(Time duration);`. ³²

`setFramerateLimit(N)` задаёт верхний предел FPS, но контроль фактически реализуется в конце кадра: после `display()` SFML делает паузу, чтобы кадр не «убежал» быстрее лимита. Это важно: если «в цикле нет `display()`», то и ограничение кадров может не сработать ожидаемо.

³³

Практический вывод для коллоквиума: если вас спрашивают «почему `setFramerateLimit(60)` работает», отвечайте «потому что SFML после `display()` измеряет `Clock` и при необходимости делает `sleep` на остаток времени кадра». ³⁴

Антиалиасинг в SFML.

В SFML антиалиасинг (MSAA) включается через `sf::ContextSettings` и поле `antiAliasingLevel` / `antialiasingLevel` (название поля зависит от версии; в SFML 3 оно документировано как `antiAliasingLevel`). Это «requested number of multisampling levels». При этом документация подчёркивает: значения — «подсказка», система может подобрать «ближайший поддерживаемый» вариант, а фактически использованные настройки можно прочитать через `Window::getSettings()`. ³⁵

Мини-пример включения MSAA:

```
sf::ContextSettings settings;
settings.antialiasingLevel = 8;
sf::RenderWindow window(sf::VideoMode(800, 600), "AA", sf::Style::Default,
settings);
```

Смысл: MSAA относится к настройкам OpenGL-контекста окна. ³⁶

Системы координат и `mapPixelToCoords/mapCoordsToPixel`.

SFML различает: - координаты **пикселей** (target coordinates) — обычно то, что вы получаете от ОС/событий мыши; - **world coordinates** («мир»), зависящие от текущего `sf::View`; - **локальные** координаты объекта (до применения transform) — объектная система координат, где «local bounds игнорируют transformations». ³⁷

`mapPixelToCoords` переводит пиксели → world coords; типичный кейс — «найти, что находится под курсором». `mapCoordsToPixel` — обратное преобразование; документация описывает, что это «обратный путь того, что делает GPU при рендере» и что при кастомном `View` или `resize` соответствие «150,75 ↔ 10,50» меняется. ³⁸

Типичная ошибка: использовать координаты мыши из событий как «мировые» координаты после того, как вы изменили `View`. Правильно: `window.mapPixelToCoords({x,y})`. ³⁹

Учебные подсказки (recall prompts): - «Почему нельзя рисовать “по-инкременту”, стирать пиксели и т.п.?» → потому что SFML ориентирован на цикл полного кадра и double buffering. ⁴⁰

- «Где реально происходит ограничение FPS?» → в `display()`, через `Clock + sleep`. ³⁴

- «Как связаны view и координаты мыши?» → `mapPixelToCoords` учитывает текущий `View`. ⁴¹

Проверка нажатий клавиш/кнопок: real-time vs events

Короткий ответ: `sf::Keyboard::isKeyPressed` / `sf::Mouse::isButtonPressed` дают **текущее** состояние устройств «в любой момент» (polling). События `KeyPressed/KeyReleased/MouseButtonPressed/...` приходят через очередь событий окна и отражают «факты» (edge-события), зависящие от фокуса и настроек key repeat. Главное различие: polling удобно для «пока удерживается — двигаться», а события — для «однократных действий» и текстового ввода. ⁴²

Подробно: что гарантирует `isKeyPressed`.

Документация SFML подчёркивает: `sf::Keyboard` позволяет «в любой момент получить состояние клавиши», не храня у себя boolean и не полагаясь на события; и это «реальное состояние», даже если окно не в фокусе и событие не пришло. ⁴³

События и очередь событий.

`pollEvent(event)` «вынимает событие с вершины очереди событий» (non-blocking) и возвращает `false`, если очередь пуста; событий может быть много, поэтому `pollEvent` надо вызывать в цикле. ⁴⁴

Типовая линия рассуждений на экзамене: - movement/камера/удержание кнопки → polling (`isKeyPressed`, `isButtonPressed`); - «нажали один раз» (toggle, клик) → события `KeyPressed`, `MouseButtonPressed`; - текстовый ввод (важно различать от `KeyPressed`) → у SFML есть `TextEntered` как отдельный тип события. ⁴⁵

Частые ошибки: - Считать, что `KeyPressed` повторяется «сам по себе одинаково везде» — на самом деле повтор зависит от режима key repeat и системных настроек. В SFML есть `setKeyRepeatEnabled`, но delay/rate определяет ОС. ⁴⁶

- Читать “не тот” член union в `sf::Event` (например, читать `event.mouseButton`, когда `event.type == KeyPressed`) — это UB. ⁴⁷

Учебные подсказки: - «Чем polling отличается от event?» → polling = состояние “сейчас”; event = “что случилось”. ⁴⁸

- «Почему SFML отдельно подчёркивает union?» → чтобы вы не читали поля, не соответствующие `event.type`. ⁴⁷

События SFML, координаты мыши, key repeat, repeat delay/rate

Короткий ответ: события SFML — это фиксированный набор типов (`Closed`, `Resized`, `KeyPressed`, `KeyReleased`, `MouseButtonPressed`, `MouseButtonReleased`, `MouseMoved`, ...), которые вы получаете из очереди событий окна через `pollEvent/waitEvent`. `setKeyRepeatEnabled(true)` включает генерацию повторяющихся `KeyPressed` при удержании клавиши; задержка до повтора и частота повтора — настройки ОС (пример: macOS «Delay until repeat» и «Key repeat rate»). ⁴⁹

Событийные типы.

Перечень ключевых типов зафиксирован в `sf::Event::EventType`: `Closed`, `Resized`, `KeyPressed`, `KeyReleased`, `MouseButtonPressed`, `MouseButtonReleased`, `MouseMoved` и др. ⁵⁰

Получение координат мыши.

В событиях типа `MouseMoved` вы получаете координаты в пикселях (относительно окна). Если вы используете `View`, для «мировых» координат применяйте `mapPixelToCoords`. ⁵¹

Key repeat и связь с OS delay/rate.

SFML управляет только тем, будут ли повторяющиеся `KeyPressed` приходить вообще: при `setKeyRepeatEnabled(false)` будет «только одно событие при нажатии». ⁵²

А вот «repeat delay» (задержка до первого повторного события) и «repeat rate»/interval (скорость повторов) задаются системой: на macOS это два ползунка «Delay until repeat» и «Key repeat rate».

Для X11/XKB это формализовано как `timeout` (delay до первого repeat) и `interval` (между subsequent repeat events). ²⁰

Вопрос-ловушка: «SFML задаёт repeat delay/rate?» — корректный ответ: SFML даёт только switch “повторять/не повторять” события; тайминги повторов определяет ОС/система ввода. ⁴⁶

Реализация простейших GUI-элементов: Draggable, Button, Slider

Здесь в билете ожидают не «фреймворк-GUI», а чистую архитектуру обработки событий + корректную математику координат + аккуратное состояние (state machine).

Общий принцип: любой интерактивный виджет удобно рассматривать как конечный автомат (idle → hover → pressed/dragging → released) и обновлять его состоянием либо (а) на событиях (`MouseMoved`, `MouseButtonPressed`, `MouseButtonReleased`), либо (б) смешанно: события для edges + polling для удержания. ⁵³

Архитектурный шаблон обработки GUI-ввода (mermaid)

```
flowchart TD
    A[Main loop] --> B[Poll events]
    B --> C{Event type?}
    C -->|MouseMoved| D[Update hover/drag position]
    C -->|MouseButtonPressed| E[Hit-test -> pressed/dragging]
    C -->|MouseButtonReleased| F[Release -> click/commit]
    C -->|Resized| G[Update view / layout]
    D --> H[Update widget states]
    E --> H
    F --> H
    G --> H
    H --> I[Update model]
    I --> J[Render: clear/draw/display]
    J --> A
```

(Идея цикла: все события вынимаем из очереди, затем один раз рендерим кадр.) ⁵⁴

Draggable (перетаскиваемая табличка)

Короткий ответ: при `MouseButtonPressed` делаем hit-test по прямоугольнику, устанавливаем `dragging=true` и запоминаем смещение (`mousePos - widgetPos`). При `MouseMoved`, если `dragging`, ставим `widgetPos = mousePos - offset`. При `MouseButtonReleased` сбрасываем `dragging`. Если есть `View`, переводим пиксельные координаты в world coords через `mapPixelToCoords`. ⁵⁵

Мини-код (псевдореализация):

```
struct Draggable {
    sf::FloatRect rect;
    bool dragging = false;
    sf::Vector2f dragOffset;
```

```

void onEvent(const sf::Event& e, sf::RenderWindow& w) {
    if (e.type == sf::Event::MouseButtonPressed &&
        e.mouseButton.button == sf::Mouse::Left) {
        auto m = w.mapPixelToCoords({e.mouseButton.x, e.mouseButton.y});
        if (rect.contains(m)) { dragging = true; dragOffset = m -
rect.getPosition(); }
    }
    if (e.type == sf::Event::MouseMove && dragging) {
        auto m = w.mapPixelToCoords({e.mouseMove.x, e.mouseMove.y});
        rect.left = m.x - dragOffset.x;
        rect.top = m.y - dragOffset.y;
    }
    if (e.type == sf::Event::MouseButtonReleased &&
        e.mouseButton.button == sf::Mouse::Left) {
        dragging = false;
    }
}
};

```

`contains` /hit-test должен работать в той же системе координат, что и `rect` (обычно world coords). ³⁹

Типичные ошибки:

(1) забыть про `dragOffset` → табличка «прыгает» так, чтобы левый верхний угол попадал в курсор; (2) сравнивать пиксельные координаты мыши с world-координатами виджета при кастомном `View`; (3) не учитывать, что события приходят только при `pollEvent`, а не «непрерывно сами». ⁵⁶

Recall prompts: «Что хранить, чтобы drag был “smooth”?» → offset между курсором и origin виджета. «Когда пересчитывать координаты?» → на drag/move событиях. ⁵¹

Button (кнопка)

Короткий ответ: кнопка — это hit-test + состояние `pressedInside`. Клик считается, если press был внутри, и release тоже внутри (иначе это drag-out/cancel). ⁵⁷

Паттерн обработки: - `MouseButtonPressed`: если внутри → `pressed=true`; - `MouseMove`: можно обновлять hover; - `MouseButtonReleased`: если `pressed` и release внутри → fire callback; затем `pressed=false`. ⁵¹

Slider (слайдер)

Короткий ответ: слайдер отображает значение `[min,max]` как позицию ползунка на треке. При drag ползунка берём `mouseX` (или проекцию на ось), нормализуем в `[0,1]`, затем `value = min + t*(max-min)`, с clamping. ⁵⁸

Как ориентир «индустриальной» формы API можно посмотреть, что у `tgui::Slider` есть `setMinimum/setMaximum` и т.п.; это помогает понять, какие методы разумно иметь в собственной реализации для коллоквиума. ⁵⁹

Типичные ошибки: - не clamp'ить значение; - некорректно учитывать ширину ползунка (thumb) при расчёте диапазона перемещения; - смешивать локальные координаты (внутри виджета) и world coords при наличии transform/view. ⁶⁰

Наследование и динамический полиморфизм в C++

Ответы по наследованию

Основы наследования, конструкторы/деструкторы, модификация доступа

Короткий ответ: производный класс содержит подобъект базового (base subobject) плюс свои поля; конструкторы базового вызываются перед конструктором производного, деструкторы — в обратном порядке. Видимость/доступ (`public/protected/private` наследование и `protected` члены) влияет на то, как базовые интерфейсы «экспортируются» наружу. ⁶¹

Для «наследования конструкторов» в C++ используется `using Base::Base;` (это другой смысл `using`, не путать с «отменой затенения»). ⁶²

Про друзей: «friend базового» не становится автоматически другом производного; доступ к приватным членам определяется дружбой относительно конкретного класса (это следует из модели доступа языка; в вопросенике это проверяют как понимание score дружбы). ⁶²

Затенение имён и `using` для отмены затенения

Короткий ответ: если в производном объявить метод/поле с тем же именем, имя из базового скрывается (name hiding). Это *не то же самое*, что перегрузка (overload): перегрузка — набор функций с одним именем, различающихся сигнатурой; затенение может «спрятать» все overload-версии базового. Типичная практика: `using Base::f;` в производном, чтобы вернуть все базовые overload'ы в область видимости. ⁶²

Приведение типов, object slicing, UB при `static_cast` вниз

Короткий ответ: upcast ($Derived \rightarrow Base$) безопасен и часто неявный; downcast ($Base \rightarrow Derived$) через `static_cast` **не делает проверок**, и безопасен только если вы *гарантируете* динамический тип объекта (иначе UB). Для безопасного downcast используйте `dynamic_cast` на полиморфном базовом типе. ⁶³

Срезка объекта (object slicing): происходит при копировании/присваивании *по значению* производного в базовый: «часть Derived отрезается», виртуальность не спасает, потому что копирование объекта базового типа создаёт *самостоятельный* base-объект. Это запрещают/подсвечивают практики вроде C++ Core Guidelines «Don't slice». ⁶⁴

Типичный учебный пример slicing:

```
struct B { virtual ~B() = default; /*...*/ };
struct D : B { int extra; };
```

```
D d;  
B b = d; // slicing: extra теряется, b — отдельный B
```

Смысл: полиморфизм работает через ссылки/указатели, а не через объекты «по значению». ⁶⁵

Строение объекта, размер, ЕВО, множественное и виртуальное множественное наследование

Короткий ответ: стандарт C++ не фиксирует точный layout объектов; компиляторы следуют ABI. Но общая идея: base subobject размещается внутри derived-объекта, и при множественном наследовании может быть несколько base subobject'ов, поэтому при приведении к разным базам указатель может «сдвигаться». ЕВО позволяет пустому базовому подобъекту занимать 0 байт (оптимизация). В виртуальном множественном наследовании для «ромба» добавляются структуры для корректной адресации виртуального базового. ⁶⁶

ЕВО (empty base optimization): базовый пустой класс может «не увеличивать размер» derived (в отличие от пустого члена-поля, который обычно ≥ 1 байт). `cppreference` прямо формулирует: «Allows the size of an empty base subobject to be zero». ⁶⁷

Сдвиг указателей при MI: если `struct C : A, B`, то адрес `C*` и `A*` может совпадать, а `B*` — быть `C* + offset`. Это и есть pointer adjustment. Такое поведение описывается ABI (например, Itanium C++ ABI подробно фиксирует «Data Layout», «Virtual Table Layout», «RTTI» и особенности виртуального наследования). ⁶⁸

Recall prompts:

- «Почему downcast static_cast опасен?» → нет runtime-проверок, UB если dynamic type не тот. ⁶⁹
- «Почему ЕВО вообще возможен?» → base subobject не обязан иметь уникальный адрес как обычный объект. ⁶⁷
- «Почему MI вызывает pointer adjustment?» → потому что разные base subobject располагаются по разным смещениям. ¹⁵

Ответы по динамическому полиморфизму

Что такое динамический полиморфизм и когда он нужен

Короткий ответ: динамический полиморфизм — это вызов переопределённых методов по динамическому типу объекта через указатель/ссылку на базовый тип; достигается виртуальными функциями (`virtual`) и поддерживается vtable/RTTI в реализациях. Он даёт позднее связывание и «плагин-подобную» расширяемость без переписывания клиентского кода. ⁷⁰

Виртуальные функции, `override/final`, виртуальный деструктор

Короткий ответ: `virtual` включает dynamic dispatch; `override` заставляет компилятор проверить, что метод действительно переопределяет базовый виртуальный; `final` запрещает дальнейшее переопределение. Для корректного удаления через `Base*` нужен виртуальный деструктор. ⁷¹

Ключевая тонкость: во время конструирования/деструкции виртуальные вызовы ведут себя «как будто объект ещё/уже не fully-derived» (в реализациях меняется «активная vtable»); поэтому

вызывать виртуальные методы из конструкторов/деструкторов — источник багов и неожиданностей. ⁶¹

Таблица виртуальных функций (vtable) и скрытый указатель

Короткий ответ: типичная реализация хранит в объекте скрытое поле (указатель на таблицу виртуальных функций), из которого при вызове `basePtr->virt()` выбирается правильный «final override». Количество таблиц определяется количеством полиморфных типов (классов/подобъектов при MI/virtual inheritance) и правилами ABI; стандарт не требует vtable, но практически компиляторы её используют. ⁶¹

Абстрактные классы, интерфейсы и `pure virtual call`

Короткий ответ: абстрактный класс содержит хотя бы одну чисто виртуальную функцию (`=0`) и не может быть инстанцирован; «интерфейс» в C++ обычно моделируют как класс без данных с чисто виртуальными методами и виртуальным деструктором. Ошибка `pure virtual call` возникает, когда попытались вызвать чисто виртуальную функцию (часто — из базового конструктора/деструктора или из неполного объекта). ⁶¹

RTTI: `static_cast`, `dynamic_cast`, `typeid`, `std::type_info`

Короткий ответ:

- `static_cast` делает преобразования без runtime-проверок; downcast безопасен только при гарантии динамического типа. ⁷²
- `dynamic_cast` делает runtime-проверку для полиморфных типов: при неудаче для указателя вернёт `nullptr`, для ссылки бросит `std::bad_cast`. ⁷³
- `typeid` возвращает `std::type_info` (объект со статическим storage), описывающий тип; `type_info` хранит «implementation-specific information», включая имя. ⁷⁴

Recall prompts:

- «Что вернёт `dynamic_cast<Derived*>(basePtr)` если `basePtr` указывает не на `Derived`?» → `nullptr`. ⁷⁵
- «Почему `dynamic_cast` требует polymorphic base?» → ему нужна RTTI/механизм виртуальности для проверки. ⁷⁶

Принципы проектирования, UML и паттерны

Принципы проектирования и SOLID

Короткий ответ: на больших кодовых базах важны модульность, сопровождаемость, тестируемость и управляемость изменений. SOLID — набор принципов проектирования классов/модулей, повышающих устойчивость к изменениям. SRP и OCP у Robert C. Martin ¹⁶ формулируются как «одна причина для изменения» (SRP) и «open for extension, closed for modification» (OCP). Исторически эти идеи связываются с тезисом David L. Parnas ⁷⁷ о том, что модули должны скрывать решения, которые вероятно изменятся. ¹⁷

SRP (single responsibility): один модуль — одна причина для изменения (Martin приводит связь с Parnas и separation of concerns). ⁷⁸

OCP (open-closed): идея идёт от Bertrand Meyer ⁷⁹; Martin пересказывает/развивает её как принцип расширяемости без изменения устойчивого интерфейса. ⁸⁰

LSP: подтипы должны быть подставимы вместо базового без нарушения корректности (семантическая подстановка; классическое формальное изложение есть у Barbara Liskov ⁸¹ и Jeannette Wing ⁸²). ⁸³

ISP: лучше несколько «узких» интерфейсов под клиентов, чем один «жирный»; помогает избегать «зависимостей от того, что не используется» (обычно обсуждают вместе с SRP на уровне интерфейсов). ⁸⁴

DIP: «high-level модули не должны зависеть от low-level; оба зависят от абстракций; абстракции не зависят от деталей» — формулировка широко приводится в русскоязычных источниках. ⁸⁵

«Инкапсулируйте то, что может измениться» (Encapsulate What Varies): в практической формулировке это означает вынести изменчивую часть за стабильный интерфейс; при этом вы либо снижаете «локализацию изменения» (SRP-аспект), либо вводите абстракцию для выбора варианта во время выполнения (часто через Strategy/State). ⁸⁶

«Предпочитайте композицию наследованию»: наследование жёстко связывает типы и делает изменения опасными (особенно при нарушении LSP); композиция позволяет строить поведение из компонентов и часто комбинируется с Decorator/Strategy/Bridge. ⁸⁷

UML-отношения: что рисовать на диаграмме классов

Вопросник требует: **наследование, зависимость, ассоциация, агрегация, композиция.**

Практическая шпаргалка (смысл в терминах кода): - **Наследование (generalization):** `class D : public B` (is-a). Риск: LSP и slicing; плюс сложный layout при MI. ⁸⁸

- **Зависимость (dependency):** «использует» (параметр функции, локальная переменная, include/импорт). DIP часто стремится обратить зависимости через интерфейсы. ⁸⁹

- **Ассоциация (association):** «имеет ссылку/указатель» на другой объект как часть состояния.

- **Агрегация:** слабое владение (например, храните `T*` / `std::weak_ptr<T>` без управления временем жизни).

- **Композиция:** сильное владение (встроенный объект-член, `std::unique_ptr`, жизненный цикл «части» неотделим от «целого»).

Поскольку UML-нотация в вопроснике не сопровождается конкретным стандартом/шаблоном, на коллоквиуме обычно достаточно корректно объяснить смысл «владение vs ссылка» и отличить агрегацию от композиции как разницу в жизненном цикле.

Паттерны: что это и зачем

Короткий ответ: паттерны — типовые решения повторяющихся задач проектирования; они дают общий словарь (Strategy, Observer, Factory Method и т.д.) и позволяют строить код, устойчивый к изменениям. Классические GoF-паттерны группируют в поведенческие/структурные/порождающие. ⁹⁰

Ниже — ответы по паттернам из списков разделов 5–7 (с фокусом на «что решает», когда применять, плюсы/минусы, типовые ловушки и мини-подсказки для запоминания).

Паттерны проектирования: поведенческие, структурные, порождающие

Поведенческие паттерны

Опора по списку: поведенческие паттерны включают Chain of Responsibility, Command, Observer, State, Strategy, Visitor и др. ⁹¹

Стратегия (Strategy)

Короткий ответ: выносит семейство алгоритмов в отдельные классы и позволяет менять алгоритм «на лету» через общий интерфейс. ⁹²

Когда применять: когда у вас `if/switch` по «режиму работы» и ожидается рост вариантов; когда нужно тестировать алгоритмы изолированно; когда выбор алгоритма может меняться во время выполнения. ⁹³

Плюсы/минусы: + заменяемость алгоритмов без изменения клиента (ОСР-эффект), + упрощение тестирования; – возможное увеличение числа классов/объектов. ⁹⁴

Кодовый скелет (как в RefactoringGuru): контекст хранит указатель/ссылку на Strategy; клиент выбирает конкретную стратегию. ⁹⁵

Mermaid-структура:

```
classDiagram
    class Context {
        -strategy: Strategy
        +setStrategy(Strategy)
        +execute()
    }
    class Strategy {
        <<interface>>
        +execute()
    }
    class ConcreteStrategyA
    class ConcreteStrategyB
    Context --> Strategy
    Strategy <|.. ConcreteStrategyA
    Strategy <|.. ConcreteStrategyB
```

Типовые ошибки:

- хранить внутри Context `enum` и снова делать `switch` вместо полиморфизма;
- «стратегия знает слишком много о контексте» → сильное зацепление, нарушение SRP/DIP. ⁹⁶

Recall prompts: «Strategy = поведение как объект». «Контекст делегирует». ⁹⁷

Цепочка обязанностей (Chain of Responsibility)

Короткий ответ: запрос проходит по цепочке обработчиков; каждый либо обрабатывает, либо передаёт дальше. ⁹¹

Когда применять: когда неизвестно, кто обработает запрос; когда вы хотите отвязать отправителя от получателя и динамически перестраивать цепочку. ⁹⁸

Плюсы/минусы: + слабая связанность, + легко добавлять обработчики; – запрос может «никем не обработаться», – сложнее отлаживать путь запроса. ⁹⁸

Типовая ловушка: «скрытая магия» — цепочки в коде без ясной сборки (composition root). ⁸⁹

Наблюдатель (Observer)

Короткий ответ: механизм подписки: издатель уведомляет подписчиков о событиях/изменениях. ⁹⁹

Publisher/Subscriber: publisher хранит список подписчиков и вызывает их `update` при событии; subscribers реализуют общий интерфейс. ⁹⁹

Когда применять: когда «изменение одного объекта должно обновлять многие другие», и набор наблюдателей динамический. GUI-события — классический пример. ¹⁰⁰

Плюсы/минусы: + ОСР (можно добавлять подписчиков без изменения издателя); – порядок уведомлений может быть важен/неочевиден; риск утечек, если не отписываться. ¹⁰⁰

Mermaid-структура:

```
classDiagram
    class Publisher {
        -subs: list<Subscriber>
        +subscribe(Subscriber)
        +unsubscribe(Subscriber)
        +notify(data)
    }
    class Subscriber {
        <<interface>>
        +update(data)
    }
    class ConcreteSubscriberA
    class ConcreteSubscriberB
    Publisher --> Subscriber : notifies
    Subscriber <|.. ConcreteSubscriberA
    Subscriber <|.. ConcreteSubscriberB
```

Команда (Command) + мультикоманда + undo

Короткий ответ: превращает действие в объект (`execute()`), позволяя ставить команды в очередь, логировать, делать undo/redo; мультикоманда — композиция команд; undo обычно реализуют как `unexecute()` / `undo()` с сохранением контекста. ⁹¹

Плюсы/минусы: + можно откладывать/повторять/отменять действия; – рост числа классов и необходимость аккуратного хранения состояния для undo. ⁹⁸

Типовая ловушка undo: если команда не сохраняет «до-состояние», undo становится невозможен или неверен.

Посетитель (Visitor)

Короткий ответ: выносит алгоритмы из структуры объектов в отдельного «посетителя», позволяя добавлять новые операции без изменения классов элементов (цена — сложность и чувствительность к изменению набора типов элементов). ¹⁰¹

Состояние (State) и конечные автоматы

Короткий ответ: объект меняет поведение при изменении внутреннего состояния так, как будто «поменял класс». Это ОО-обёртка над конечным автоматом (FSM). ¹⁰¹

Когда применять: когда много ветвлений `if(state==...)` и поведение зависит от состояния; особенно в UI/играх (idle/run/jump и т.п.). ¹⁰²

Структурные паттерны

Опора по списку: Adapter, Facade, Proxy, Decorator, Composite, Bridge. ¹⁰³

Адаптер (Adapter)

Короткий ответ: приводит несовместимый интерфейс к ожидаемому; полезен при интеграции legacy/сторонних библиотек. ¹⁰⁴

Фасад (Facade)

Короткий ответ: даёт упрощённый высокоуровневый интерфейс к сложной подсистеме; снижает coupling клиента с деталями. ¹⁰⁴

Заместитель (Proxy)

Короткий ответ: контролирует доступ к объекту-реальному субъекту (lazy load, remote proxy, protection proxy, caching, logging). ¹⁰⁴

Декоратор (Decorator) и отличие от Adapter/Proxy

Короткий ответ: добавляет поведение динамически, оборачивая объект в цепочку «обёрток», сохраняя интерфейс. ¹⁰⁴

Отличие: - Decorator меняет/расширяет поведение при сохранении интерфейса;
- Adapter меняет интерфейс;
- Проху контролирует доступ, часто сохраняя интерфейс, но мотивация — не «добавить фичи», а «управлять объектом». 104

Компоновщик (Composite)

Короткий ответ: древовидная структура «часть-целое»; позволяет работать с группой объектов как с одним. 104

Мост (Bridge) и отличие от Strategy

Короткий ответ: разделяет «абстракцию» и «реализацию» на две иерархии, чтобы независимо изменять обе. 104

Bridge vs Strategy: оба используют композицию, но цель различается: Strategy — смена алгоритма/поведения у контекста; Bridge — разрыв жёсткой связки «абстракция ↔ реализация» (обычно считается структурной перестройкой архитектуры, а не выбором алгоритма). 105

Порождающие паттерны

Опора по списку: Factory Method, Abstract Factory, Builder, Prototype, Singleton. 106

Фабричный метод (Factory Method)

Короткий ответ: делегирует создание объектов подклассам/переопределяемому методу, сохраняя общий интерфейс создания. 106

Абстрактная фабрика (Abstract Factory)

Короткий ответ: создаёт семейства связанных объектов без привязки к конкретным классам; часто применяется для кроссплатформенных UI-наборов, тем, драйверов. 106

Строитель (Builder)

Короткий ответ: пошагово собирает сложный объект; позволяет разные представления при одинаковом «процессе строительства». 106

Прототип (Prototype)

Короткий ответ: копирует существующие объекты без знания их конкретных классов (полезно, когда создание дорого или классы задаются динамически). 106

Синглтон (Singleton)

Короткий ответ: гарантирует один экземпляр и глобальную точку доступа; на практике требует осторожности (глобальное состояние, тестируемость, порядок инициализации). 106

Типовая реализация (на коллоквиум): `static` локальный объект в `instance()` (ленивая инициализация). Важно уметь обсудить потоко-безопасность (в C++11 и выше инициализация

function-local static синхронизирована), но архитектурная критика синглтона может принести дополнительные баллы.

Приложение: опорные «карточки» для заучивания

Ниже — компактные prompts, которые удобно проговаривать вслух.

SFML - «События: где они лежат?» → в очереди окна, вынимаются `pollEvent`. 107
- «Почему `sf::Event` опасен?» → это union, читать нужно только поле по `event.type`. 47
- «Координаты мыши + View?» → `mapPixelToCoords`. 41
- «Double buffering?» → `display()` переносит из hidden buffer на окно. 40

Наследование/полиморфизм - «static_cast вниз» → без проверок, UB если тип не тот. 72
- «dynamic_cast» → runtime check; `nullptr` / `bad_cast`. 73
- «Slicing» → копирование Derived в Base по значению. 65
- «EBO» → пустой базовый может занимать 0 байт. 67

SOLID/паттерны - «SRP» → одна причина изменения. 78
- «ОСР» → расширяем без модификации стабильного интерфейса. 80
- «Strategy vs Bridge» → выбор алгоритма vs разрыв абстракции/реализации. 87

1 18 91 <https://refactoring.guru/ru/design-patterns/behavioral-patterns>
<https://refactoring.guru/ru/design-patterns/behavioral-patterns>

2 37 38 39 41 56 58 https://www.sfml-dev.org/documentation/2.6.1/classsf_1_1RenderWindow.php
https://www.sfml-dev.org/documentation/2.6.1/classsf_1_1RenderWindow.php

3 47 <https://www.sfml-dev.org/tutorials/2.6/window-events.php>
<https://www.sfml-dev.org/tutorials/2.6/window-events.php>

4 16 17 25 78 86 96 <https://blog.cleancoder.com/uncle-bob/2014/05/08/SingleReponsibilityPrinciple.html>
[SingleReponsibilityPrinciple.html](https://blog.cleancoder.com/uncle-bob/2014/05/08/SingleReponsibilityPrinciple.html)

5 89 <https://technical-excellence.ru/wiki/DependencyInversionPrinciple.html>
<https://technical-excellence.ru/wiki/DependencyInversionPrinciple.html>

6 14 24 61 62 70 71 <https://en.cppreference.com/w/cpp/language/virtual.html>
<https://en.cppreference.com/w/cpp/language/virtual.html>

7 8 <https://github.com/SFML/SFML>
<https://github.com/SFML/SFML>

9 26 <https://github.com/RefactoringGuru/design-patterns-cpp>
<https://github.com/RefactoringGuru/design-patterns-cpp>

10 <https://github.com/texus/TGUI>
<https://github.com/texus/TGUI>

11 27 28 29 40 <https://www.sfml-dev.org/tutorials/2.6/graphics-draw.php>
<https://www.sfml-dev.org/tutorials/2.6/graphics-draw.php>

12 42 43 45 48 https://www.sfml-dev.org/documentation/2.6.0/classsf_1_1Keyboard.php
https://www.sfml-dev.org/documentation/2.6.0/classsf_1_1Keyboard.php

13 23 63 69 72 https://www.cppreference.com/w/cpp/language/static_cast.html
https://www.cppreference.com/w/cpp/language/static_cast.html

15 66 68 <https://itanium-cxx-abi.github.io/cxx-abi/abi.html>
<https://itanium-cxx-abi.github.io/cxx-abi/abi.html>

19 Set how quickly a key repeats on Mac - Apple Palaikymas (LT)
https://support.apple.com/lt-lt/guide/mac-help/mchl0311bdb4/mac?utm_source=chatgpt.com

20 The X Keyboard Extension:
https://www.x.org/releases/X11R7.7/doc/libX11/XKB/xkblib.html?utm_source=chatgpt.com

21 44 54 107 https://www.sfml-dev.org/documentation/2.6.1/classsf_1_1WindowBase.php
https://www.sfml-dev.org/documentation/2.6.1/classsf_1_1WindowBase.php

22 49 50 51 53 55 57 https://www.sfml-dev.org/documentation/2.6.0/classsf_1_1Event.php
https://www.sfml-dev.org/documentation/2.6.0/classsf_1_1Event.php

30 Documentation for SFML 3.0.0 - Simple and Fast Multimedia Library
https://www.sfml-dev.org/documentation/3.0.0/classsf_1_1Time.html?utm_source=chatgpt.com

31 SFML - Simple and Fast Multimedia Library - SFML Documentation
https://documentation.help/SFML/classsf_1_1Clock.html?utm_source=chatgpt.com

32 Documentation for SFML 3.0.0 - Simple and Fast Multimedia Library
https://www.sfml-dev.org/documentation/3.0.1/Sleep_8hpp_source.html?utm_source=chatgpt.com

33 34 <https://stackoverflow.com/questions/74248066/how-does-sfml-internally-maintain-the-frame-rate-limit-when-setframeratelimit>
<https://stackoverflow.com/questions/74248066/how-does-sfml-internally-maintain-the-frame-rate-limit-when-setframeratelimit>

35 https://www.sfml-dev.org/documentation/3.0.0/structsf_1_1ContextSettings.html
https://www.sfml-dev.org/documentation/3.0.0/structsf_1_1ContextSettings.html

36 https://www.sfml-dev.org/documentation/2.1/structsf_1_1ContextSettings.php
https://www.sfml-dev.org/documentation/2.1/structsf_1_1ContextSettings.php

46 52 77 https://www.sfml-dev.org/documentation/2.6.0/classsf_1_1WindowBase.php
https://www.sfml-dev.org/documentation/2.6.0/classsf_1_1WindowBase.php

59 https://tgui.eu/documentation/latest-stable/classtgui_1_1Slider.html
https://tgui.eu/documentation/latest-stable/classtgui_1_1Slider.html

60 sf::Text Class Reference (SFML / Learn / 2.6.2 Documentation)
https://en.sfml-dev.org/documentation/2.6.2/classsf_1_1Text.php?utm_source=chatgpt.com

64 65 84 <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines?lang=en>
<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines?lang=en>

67 <https://en.cppreference.com/w/cpp/language/ebo.html>
<https://en.cppreference.com/w/cpp/language/ebo.html>

73 75 76 https://www.cppreference.com/w/cpp/language/dynamic_cast.html
https://www.cppreference.com/w/cpp/language/dynamic_cast.html

74 <https://en.cppreference.com/w/cpp/language/typeid>
<https://en.cppreference.com/w/cpp/language/typeid>

79 106 <https://refactoring.guru/ru/design-patterns/creational-patterns>
<https://refactoring.guru/ru/design-patterns/creational-patterns>

80 <https://blog.cleancoder.com/uncle-bob/2014/05/12/TheOpenClosedPrinciple.html>
<https://blog.cleancoder.com/uncle-bob/2014/05/12/TheOpenClosedPrinciple.html>

81 99 <https://refactoring.guru/ru/design-patterns/observer>
https://refactoring.guru/ru/design-patterns/observer

82 103 104 105 <https://refactoring.guru/ru/design-patterns/structural-patterns>
https://refactoring.guru/ru/design-patterns/structural-patterns

83 https://en.wikipedia.org/wiki/Liskov_substitution_principle
https://en.wikipedia.org/wiki/Liskov_substitution_principle

⁸⁵ [https://ru.wikipedia.org/wiki/
%D0%9F%D1%80%D0%B8%D0%BD%D1%86%D0%B8%D0%BF_%D0%B8%D0%BD%D0%B2%D0%B5%D1%80%D1%81%D0%
https://ru.wikipedia.org/wiki/%D0%9F%D1%80%D0%B8%D0%BD%D1%86%D0%B8%D0%BF_%D0%B8%D0%BD%D0%B2%D0%B5%D1%80%D1%81%](https://ru.wikipedia.org/wiki/%D0%9F%D1%80%D0%B8%D0%BD%D1%86%D0%B8%D0%BF_%D0%B8%D0%BD%D0%B2%D0%B5%D1%80%D1%81%D0%D1%8A%D0%BA%D0%BB%D0%BC%D0%BD%D0%BE%D0%BF%D0%B0%D0%B1%D0%B2%D0%B3%D0%B4%D0%B5%D0%B6%D0%B7%D0%B8%D0%B9%D0%CA%D0%CB%D0%CC%D0%CD%D0%CE%D0%CF%D0%D0%D1%D2%D3%D4%D5%D6%D7%D8%D9%DE%DF%E0%E1%E2%E3%E4%E5%E6%E7%E8%E9%FA%FB%FC%FD%FE%FF)

87 92 93 94 95 97 <https://refactoring.guru/ru/design-patterns/strategy>
<https://refactoring.guru/ru/design-patterns/strategy>

88 <https://stackoverflow.com/questions/274626/what-is-object-slicing>
<https://stackoverflow.com/questions/274626/what-is-object-slicing>

90 <https://refactoring.guru/ru/design-patterns>
https://refactoring.guru/ru/design-patterns

98 101 102 <https://refactoring.guru/design-patterns/behavioral-patterns>
https://refactoring.guru/design-patterns/behavioral-patterns

100 <https://refactoring.guru/design-patterns/observer>
https://refactoring.guru/design-patterns/observer